



**INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO**



Ingeniería en Sistemas Computacionales

SISTEMAS DISTRIBUIDOS

Nombre:

Ávila Cigarroa Aarón Axel

Profesor:

Carreto Arellano Chadwick

Práctica 7: “PWA”

Contenido

Antecedentes.....	3
Problemática	4
Solución	4
Implementación	5
Resultados.....	9
Ilustración 1. Ejecución de servidor Express en Node.js.....	9
Ilustración 2. Instalación de dependencias	9
Ilustración 3. Ejecución de servidor estático	9
Ilustración 4. Instalación de aplicación en dispositivo	10
Ilustración 5. Aplicación instalada correctamente	11
Ilustración 6. Demostración de instalación en dispositivo	12
Ilustración 7. Cambio de red a modo sin conexión en opciones de desarrollo.....	13
Ilustración 8. Visualización de aplicación en modo sin conexión.....	14
Ilustración 9 Visualizar mapa en mado sin conexión I	15
Ilustración 10 Visualizar mapa en mado sin conexión II.....	16
Ilustración 11. Intento de reserva en modo sin conexión	17
Ilustración 12. Peticiones HTTP.....	18
Conclusiones.....	18
Anexos	19

Antecedentes

En la Práctica 6, el sistema completó su migración hacia una arquitectura orientada a **Servicios Web (API RESTful)**, logrando una separación total entre la lógica de negocio (Backend en Node.js) y la interfaz de usuario (Frontend en HTML/JS). Esto garantizó la interoperabilidad del sistema; sin embargo, heredó la principal debilidad de las aplicaciones web tradicionales: la dependencia absoluta y continua de una conexión a internet.

Para que un sistema distribuido alcance la madurez comercial, debe ser capaz de ofrecer una experiencia fluida, instalable y resiliente ante fallos de red, características que históricamente estaban reservadas para las aplicaciones nativas (desarrolladas específicamente para iOS, Android o Windows).

En 2015, el diseñador Frances Berriman y el ingeniero de Google Alex Russell acuñaron el término "**Progressive Web Apps**" (PWA) para describir una nueva clase de aplicaciones que aprovechan las nuevas capacidades de los navegadores modernos para ofrecer una experiencia de usuario idéntica a la nativa. Las PWA no son un nuevo lenguaje de programación o un framework, sino un conjunto de estándares y mejores prácticas de la W3C. La revolución de las PWA fue posible gracias a la introducción de los **Service Workers**, un script que el navegador ejecuta en segundo plano, separado de la página web, abriendo la puerta a funcionalidades que no necesitan una página web abierta o interacción del usuario, como la interceptación de red (Proxy del lado del cliente) y el almacenamiento en caché avanzado. Hoy en día, plataformas como X (Twitter), Spotify y Uber utilizan PWA para ofrecer aplicaciones ligeras, instalables e independientes de la red.

Problemática

A pesar de contar con una arquitectura backend robusta, el cliente web actual presenta vulnerabilidades críticas en la capa de presentación y experiencia de usuario (UX):

1. **Falta de Tolerancia a Fallos de Red (Offline):** Si el servidor se cae o el estudiante pierde la conexión a internet de la universidad, el navegador muestra el error genérico de "No hay conexión" (el clásico juego del dinosaurio). El sistema colapsa visualmente, impidiendo que el usuario al menos consulte el menú o reciba información contextual sobre la falla.
2. **Experiencia de Usuario No Nativa:** El sistema debe ejecutarse obligatoriamente dentro de la interfaz de un navegador (con barras de búsqueda, pestañas, etc.), lo cual reduce el espacio visual y la inmersión, impidiendo que la herramienta se perciba como un software de escritorio o móvil oficial de la institución.
3. **Manejo de Errores Frágil:** En la práctica anterior, la falta de comunicación con la API resultaba en errores técnicos en la consola (TypeError) al intentar iterar sobre respuestas indefinidas.

Solución

Para mitigar la dependencia de la red y dotar al cliente web de características nativas, se propone transformar la aplicación web en una **Progressive Web App (PWA)** estandarizada mediante la implementación de las siguientes tecnologías:

- **Manifiesto de Aplicación Web (Web App Manifest):** Se integrará un archivo de metadatos en formato JSON que dictará al sistema operativo cómo debe comportarse la aplicación al ser instalada (iconos, nombre corto, colores de tema y modo de visualización sin bordes o *standalone*).
- **Service Worker como Proxy de Red:** Se programará un script intermediario entre el navegador y la red. Este interceptará todas las peticiones salientes (eventos *fetch*).
- **Estrategia "Cache-First" para Archivos Estáticos:** El HTML, CSS y JavaScript principal se descargarán una única vez y se guardarán en el almacenamiento persistente del navegador (Caché API). En futuras visitas, el sistema cargará instantáneamente desde el disco duro del cliente, incluso sin internet.
- **Manejo Amigable de Caídas de API:** Si el Service Worker detecta que la API RESTful (Backend) no responde por falta de red, en lugar de permitir que la página colapse, inyectará un objeto JSON de respuesta personalizado para que el frontend despliegue una ventana emergente (Modal) notificando amigablemente el modo *Offline*.

Implementación

El backend (*server.js* y *RoomManager.js*) se mantuvo sin modificaciones, demostrando que la evolución a PWA es un concepto puramente del lado del cliente. Las modificaciones en la carpeta frontend abarcaron:

A. Registro y Metadatos:

- **manifest.json:** Archivo creado para definir la identidad de la aplicación. Se configuró la propiedad "display": "standalone" para eliminar la barra del navegador, junto con la paleta de colores institucional (azul y blanco) y los iconos requeridos por los sistemas operativos.

```
{
  "name": "Reservas ESCOM",
  "short_name": "ESCOM",
  "description": "Sistema de Reservas Universitarias PWA",
  "start_url": "./index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#004b93",
  "icons": [
    {
      "src": "https://cdn-icons-png.flaticon.com/512/1944/1944200.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

- **index.html:** Se agregaron las etiquetas <link rel="manifest"> y <meta name="theme-color"> en la cabecera para vincular el manifiesto al ciclo de vida del DOM.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Reservas ESCOM - PWA</title>
  <link rel="stylesheet" href="style.css">

  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#004b93">
  <link rel="apple-touch-icon" href="https://cdn-icons-
png.flaticon.com/512/1944/1944200.png">
</head>
```

B. El Service Worker (sw.js): Se implementó el ciclo de vida estándar de un Service Worker:

1. **Evento install:** Descarga y guarda en el caché local todos los *assets* críticos (index.html, style.css, app.js).

```
self.addEventListener('install', event => {
  // skipWaiting() fuerza a que el nuevo SW se active inmediatamente
  self.skipWaiting();
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => {
        console.log("Caché abierto correctamente. Guardando
assets...");
        return cache.addAll(ASSETS_TO_CACHE);
      })
      .catch(error => {
        console.error("Error al guardar en caché durante la
instalación:", error);
      })
  );
});
```

2. **Evento activate:** Diseñado para gestionar el versionamiento, eliminando cachés antiguos cuando se actualiza la interfaz.

```
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.map(cacheName => {
          if (cacheName !== CACHE_NAME) {
            console.log("Limpiando caché antiguo:",
cacheName);
            return caches.delete(cacheName);
          }
        })
      );
    })
  );
  self.clients.claim(); // Tomar control de todos los clientes
inmediatamente
});
```

3. Evento fetch (El Enrutador Inteligente): Discrimina el tráfico en dos rutas lógicas:

- **Tráfico de Interfaz (Archivos):** Intercepta la petición y responde inmediatamente con la versión almacenada en caché (`caches.match`). Si falla, acude a la red.

```
event.respondWith(  
  caches.match(event.request)  
  .then(cachedResponse => {  
    if (cachedResponse) {  
      return cachedResponse; // Si está en caché, lo devolvemos  
    }  
  
    // Si no está en caché, intentamos obtenerlo de la red  
    return fetch(event.request).then(networkResponse => {  
      return networkResponse;  
    }).catch(error => {  
      console.error("Fallo al buscar en caché y en red:",  
event.request.url, error);  
  
      // Como último recurso, si pedían una página y no hay  
red, podríamos devolver el index  
      if (event.request.mode === 'navigate') {  
        return caches.match('/index.html');  
      }  
  
      throw error; // Lanza el error para que el navegador sepa  
que falló  
    });  
  })  
);
```

- **Tráfico de Datos (Rutas /api/):** Intenta alcanzar el servidor backend. Si la promesa de red es rechazada (ej. equipo desconectado), el Service Worker sintetiza una respuesta HTTP 503 local (`new Response(...)`) con un mensaje JSON controlado: *"Estás en modo sin conexión"*.

```
if (url.pathname.startsWith('/api/')) {  
  event.respondWith(  
    fetch(event.request).catch(() => {  
      // Si la API falla (estamos offline), devolvemos el JSON  
de error  
  
      return new Response(JSON.stringify({  
        success: false,
```

```

        message: "Estás en modo sin conexión. No puedes hacer
reservas ahora mismo."
      )), {
        headers: { 'Content-Type': 'application/json' },
        status: 503 // Servicio no disponible
      });
    })
  );
  return; // Termina la ejecución para peticiones de API
}

```

C. Refactorización del Cliente (app.js):

- Se añadió un bloque inicial para registrar el archivo sw.js mediante la API `navigator.serviceWorker.register()`.

```

if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('./sw.js')
      .then(registration => {
        console.log('Service Worker registrado con éxito:',
registration.scope);
      })
      .catch(error => {
        console.log('Fallo al registrar el Service Worker:',
error);
      });
  });
}

```

- Se implementaron validaciones lógicas en las funciones asíncronas (`cargarSalas` y `consultarMapa`) para verificar que el *payload* recibido sea válido (arreglos de datos reales). Si se detecta el mensaje de error sintetizado por el Service Worker, se detiene la ejecución del renderizado y se invoca la función `mostrarAlerta(mensaje, "error")` utilizando el Modal de diseño estético, evitando errores de tipo `TypeError` en consola.

Resultados

A diferencia de las pruebas en prácticas anteriores, donde se muestra la funcionalidad del proyecto, esta vez se mostrará la validación del comportamiento PWA. No se evalúa el código, sino el ciclo de vida de la aplicación en el dispositivo final:

1. Preparativos

- Para nuestro caso, nos aseguramos de tener Node.js instalado en nuestra computadora. Abrimos la terminal y levantamos el backend (el servidor RESTful). Necesitamos que la lógica de reservaciones esté corriendo para que el frontend tenga con quién comunicarse.

```
PS D:\Documentos\ESCOM\7 Sistemas Distribuidos\Práctica 7\P7_PWA\backend> node server.js
Servidor de Web Services escuchando en http://localhost:5000
```

Ilustración 1. Ejecución de servidor Express en Node.js

- Ahora levantamos el frontend, que sería el servidor estático de la PWA. Para que el navegador detecte el archivo *manifest.json* y el *sw.js*, necesitamos un servidor web ligero para la carpeta del frontend. Instalamos una dependencia en Node con npm:

```
PS D:\Documentos\ESCOM\7 Sistemas Distribuidos\Práctica 7\P7_PWA> npm install -g serve

added 85 packages in 6s

26 packages are looking for funding
  run `npm fund` for details
PS D:\Documentos\ESCOM\7 Sistemas Distribuidos\Práctica 7\P7_PWA>
```

Ilustración 2. Instalación de dependencias

Después de instalar la dependencia, podemos ejecutar el servidor estático.

```
PS D:\Documentos\ESCOM\7 Sistemas Distribuidos\Práctica 7\P7_PWA\frontend> serve

Serving!

- Local:   http://localhost:3000
- Network: http://169.254.80.190:3000

Copied local address to clipboard!
```

Ilustración 3. Ejecución de servidor estático

2. **Prueba de Instalación Nativa:** Al acceder a la ruta local del frontend a través de un navegador moderno, el sistema reconoció el archivo *manifest.json* y el Service Worker activo, habilitando el botón de instalación (Add to Home Screen). Se corroboró que la aplicación se instala en el sistema operativo, creando un acceso directo y ejecutándose en una ventana independiente sin controles de navegador (Standalone), idéntica a un software de escritorio nativo.

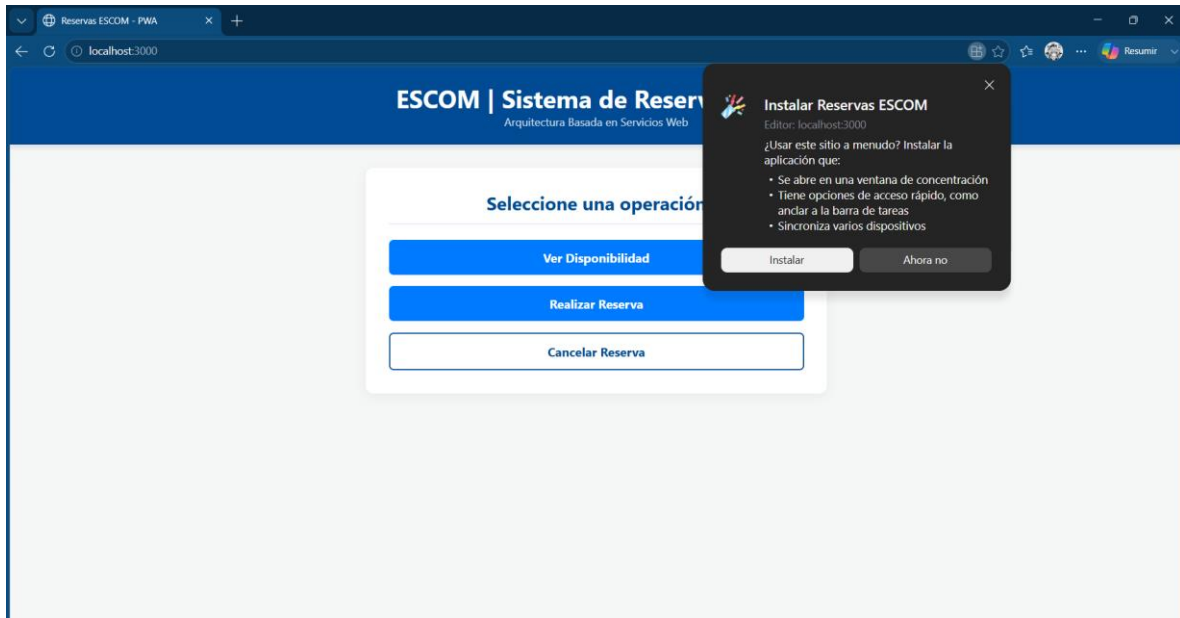


Ilustración 4. Instalación de aplicación en dispositivo

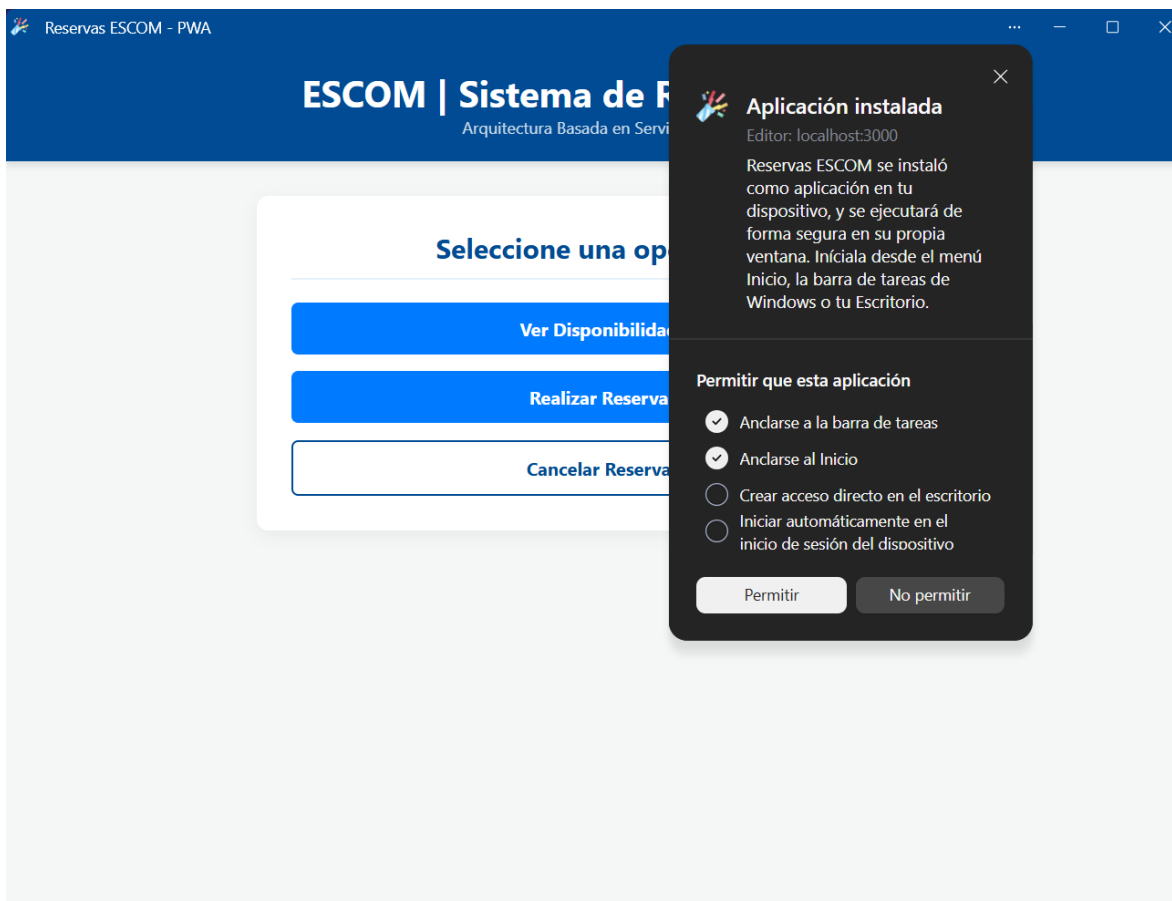


Ilustración 5. Aplicación instalada correctamente

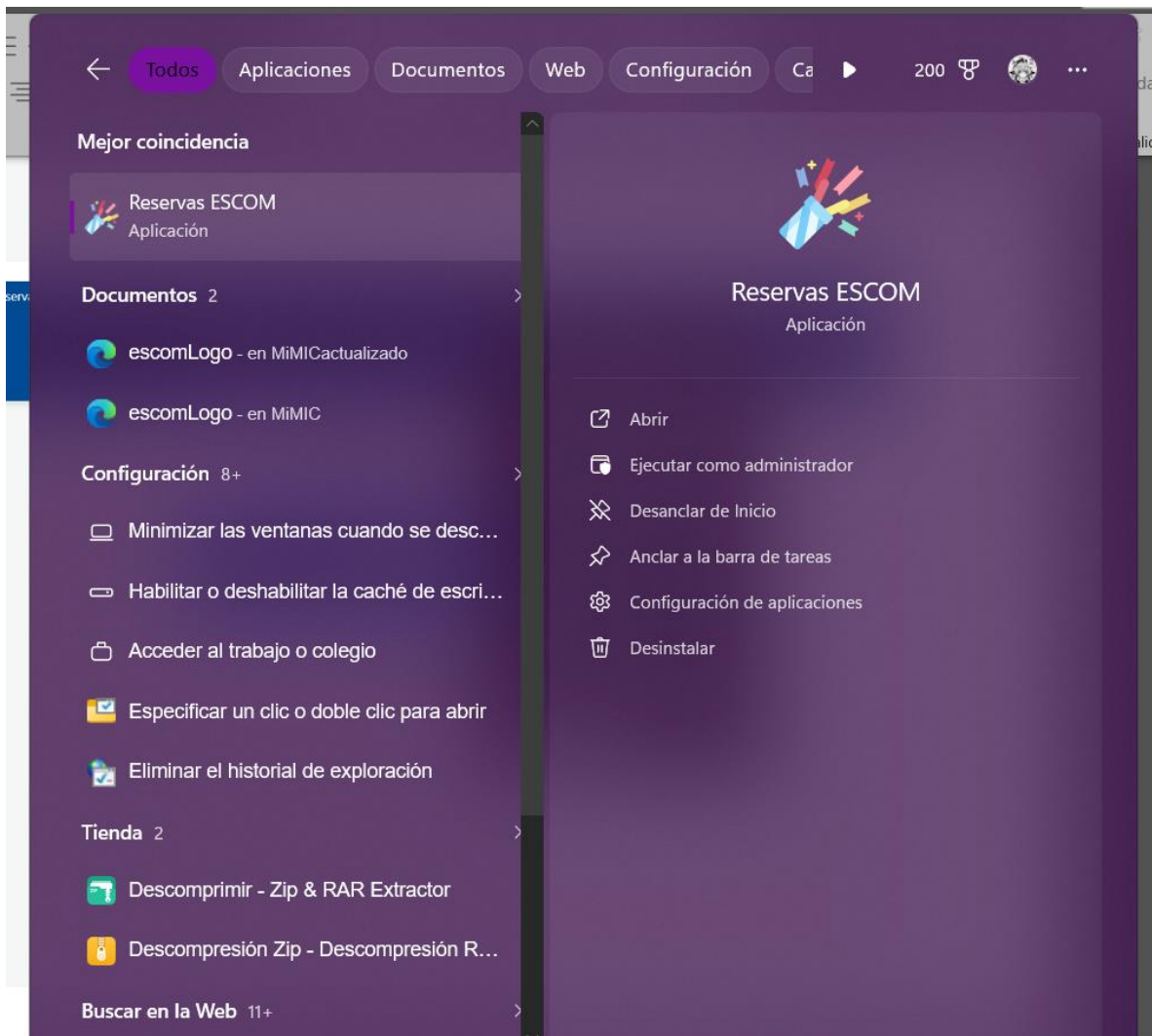


Ilustración 6. Demostración de instalación en dispositivo

- 3. Prueba de Tolerancia a Fallos (Modo Offline):** Se simuló una desconexión total de red (utilizando las herramientas de desarrollador del navegador) y se recargó la aplicación. El sistema no mostró la pantalla de error del navegador; en su lugar, cargó la interfaz gráfica completa y estilizada en milisegundos gracias al Caché API.

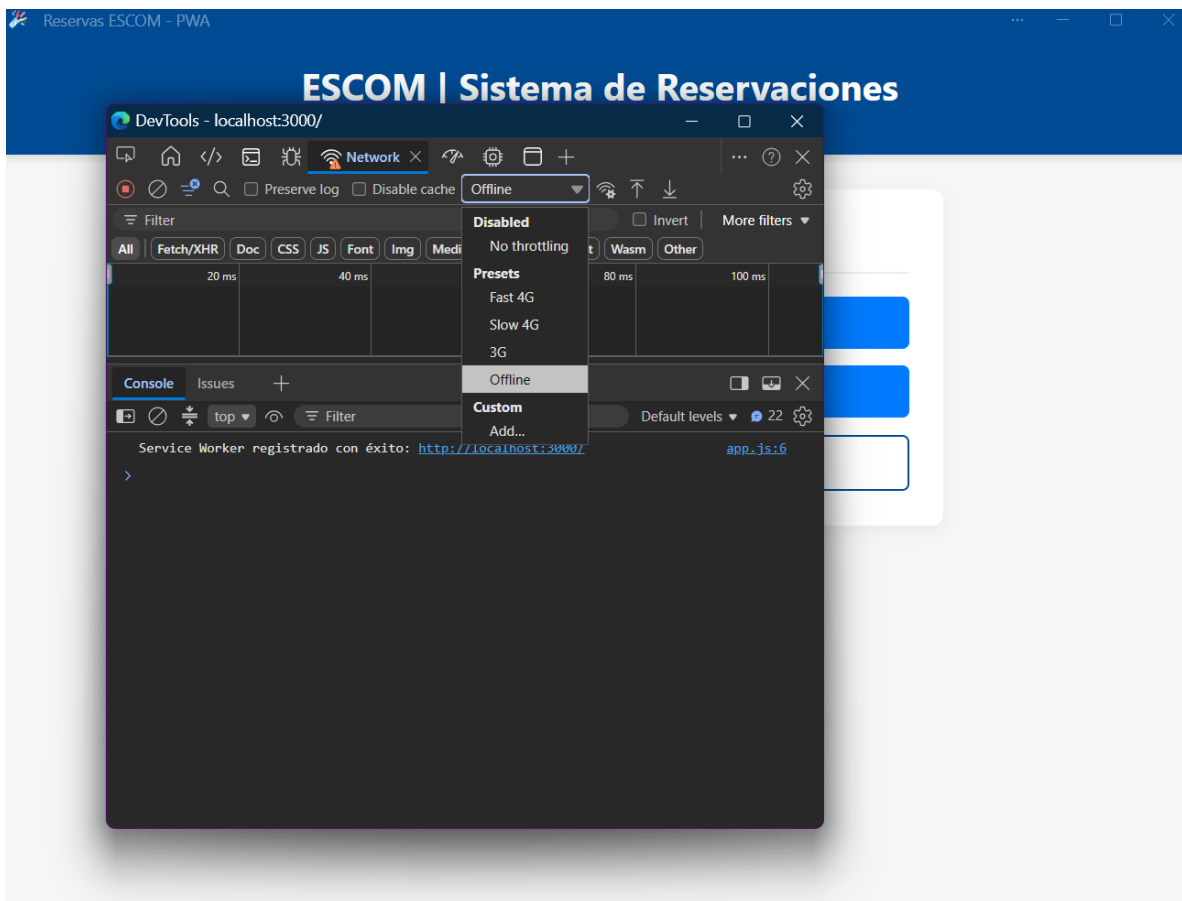


Ilustración 7. Cambio de red a modo sin conexión en opciones de desarrollo

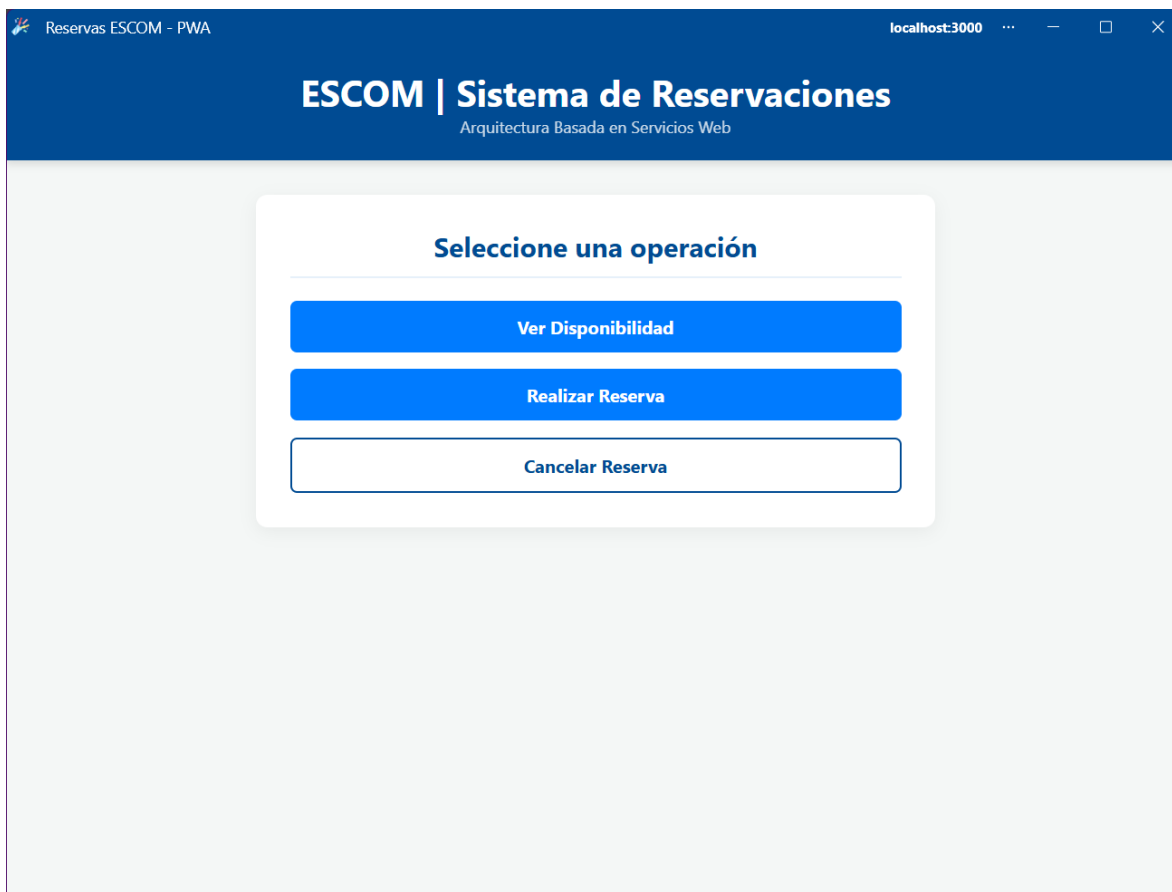


Ilustración 8. Visualización de aplicación en modo sin conexión

Reservas ESCOM - PWA

...

—

□

×

ESCOM | Sistema de Reservas
Arquitectura Basada en Servicios Web

Consulta de Disponibilidad

Modo sin conexión (Salas no disponibles) ▾

Lunes ▾

8:00 hrs ▾

Buscar

Libre

Ocupado

Volver al Menú

Ilustración 9 Visualizar mapa en mado sin conexión I

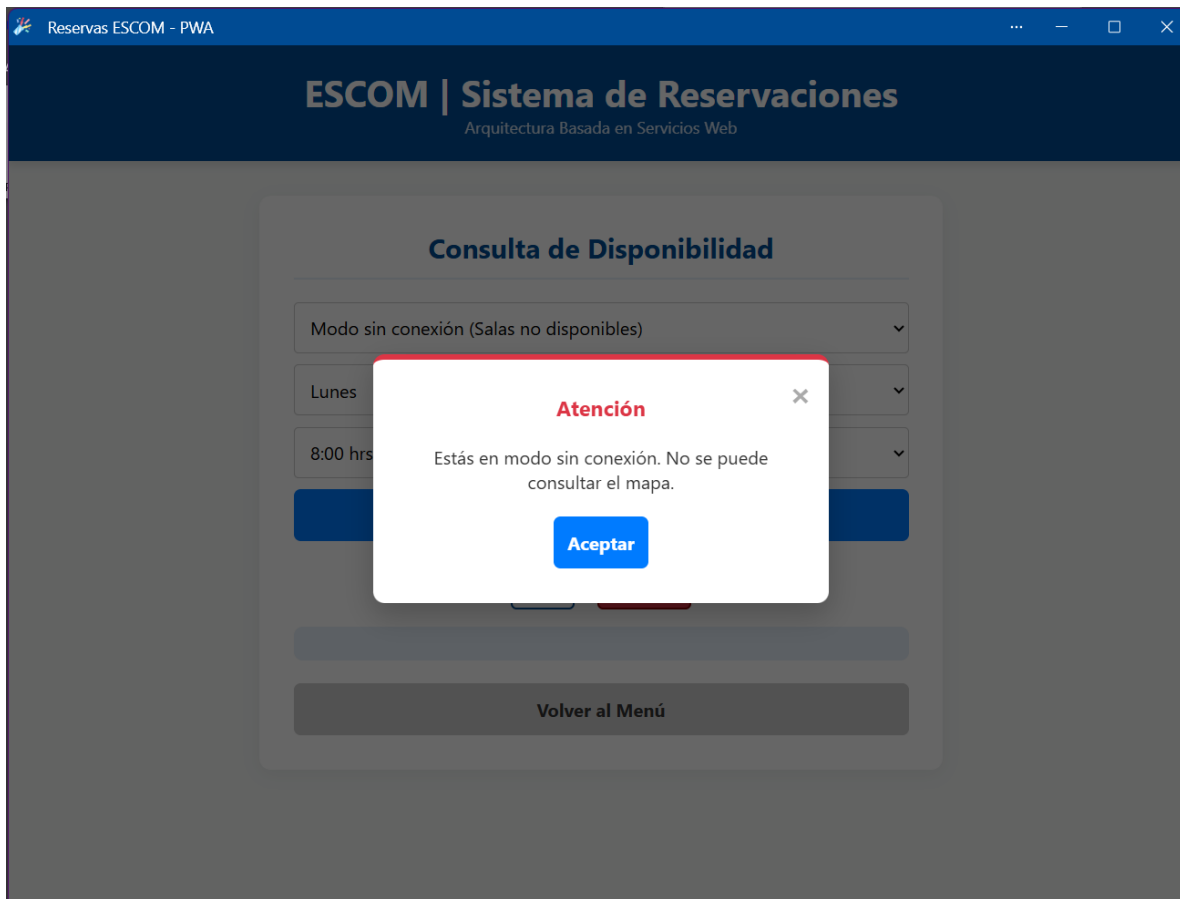


Ilustración 10 Visualizar mapa en modo sin conexión II

4. **Resiliencia Transaccional:** Estando en modo Offline, se intentó procesar una reservación de asiento. El Service Worker interceptó la caída de la petición, devolviendo el JSON simulado. El frontend leyó este objeto, previno el colapso de la vista y desplegó la ventana modal informando correctamente al usuario: "Estás en modo sin conexión. No puedes hacer reservas ahora mismo." Los menús desplegables se adaptaron indicando la indisponibilidad de datos.

Reservas ESCOM - PWA

ESCOM | Sistema de Reservas

Arquitectura Basada en Servicios Web

Realizar Reservación

Sala:

Modo sin conexión (Salas no disponibles) ▼

Asiento (1-36):

4

Día:

Lunes ▼

Hora de Inicio:

8:00 hrs ▼

Duración (horas):

1

Confirmar Acción

Volver al Menú

! Selecciona un elemento de la lista.

Ilustración 11. Intento de reserva en modo sin conexión

Finalmente, se puede visualizar en la consola del servidor estático del frontend las peticiones realizadas con las pruebas hechas:

```
HTTP 22/6/2026 1:27:57 a.m. ::1 GET /
HTTP 22/6/2026 1:27:57 a.m. ::1 Returned 304 in 261 ms
HTTP 22/6/2026 1:27:59 a.m. ::1 GET /sw.js
HTTP 22/6/2026 1:27:59 a.m. ::1 Returned 200 in 31 ms
HTTP 22/6/2026 1:27:59 a.m. ::1 GET /
HTTP 22/6/2026 1:27:59 a.m. ::1 GET /style.css
HTTP 22/6/2026 1:27:59 a.m. ::1 Returned 200 in 2 ms
HTTP 22/6/2026 1:27:59 a.m. ::1 Returned 304 in 3 ms
HTTP 22/6/2026 1:27:59 a.m. ::1 GET /
HTTP 22/6/2026 1:27:59 a.m. ::1 Returned 304 in 1 ms
HTTP 22/6/2026 1:27:59 a.m. ::1 GET /app.js
HTTP 22/6/2026 1:27:59 a.m. ::1 GET /manifest.json
HTTP 22/6/2026 1:27:59 a.m. ::1 Returned 304 in 2 ms
HTTP 22/6/2026 1:27:59 a.m. ::1 Returned 304 in 3 ms
HTTP 22/6/2026 1:31:14 a.m. ::1 GET /.well-known/appspecific/com.chrome.devtools.json
HTTP 22/6/2026 1:31:14 a.m. ::1 Returned 404 in 3 ms
HTTP 22/6/2026 1:31:57 a.m. ::1 GET /sw.js
HTTP 22/6/2026 1:31:57 a.m. ::1 Returned 304 in 4 ms
HTTP 22/6/2026 1:32:18 a.m. ::1 GET /.well-known/appspecific/com.chrome.devtools.json
HTTP 22/6/2026 1:32:18 a.m. ::1 Returned 404 in 3 ms
HTTP 22/6/2026 1:32:34 a.m. ::1 GET /.well-known/appspecific/com.chrome.devtools.json
HTTP 22/6/2026 1:32:34 a.m. ::1 Returned 404 in 2 ms
HTTP 22/6/2026 1:32:35 a.m. ::1 GET /sw.js
HTTP 22/6/2026 1:32:35 a.m. ::1 Returned 304 in 2 ms
```

Ilustración 12. Peticiones HTTP

Conclusiones

La implementación de la **Progressive Web App** representa la culminación exitosa en la evolución del lado del cliente dentro de este proyecto de Sistemas Distribuidos. Se logró transformar un cliente dependiente y frágil en una entidad robusta, instalable y con un alto grado de disponibilidad visual independiente del estado de la red.

El uso estratégico del **Service Worker** fungió como un proxy inverso instalado directamente en la computadora del usuario, demostrando cómo se pueden aplicar los conceptos de enrutamiento y tolerancia a fallos en el "borde" de la red (*Edge Computing*). Las respuestas simuladas (Mocking) durante los eventos de pérdida de red aseguraron que la aplicación fallara "con gracia" (*Graceful Degradation*), protegiendo la experiencia del usuario y previniendo excepciones en el código JavaScript.

Con esta práctica, el sistema ha transitado por todo el espectro de la evolución de arquitecturas de software: iniciando con la resolución local de condiciones de carrera mediante memoria compartida (Multiprocesamiento), abstrayendo la red con Sockets TCP y Java RMI (Objetos Distribuidos), dividiendo el negocio en Microservicios escalables (API RESTful), y finalizando con una distribución inteligente del almacenamiento y la interfaz mediante PWA. El resultado es un ecosistema tecnológico moderno, altamente interoperable y tolerante a fallos desde el núcleo del servidor hasta el dispositivo final del estudiante.

Anexos

Repositorio a GitHub con los archivos de esta práctica:

https://github.com/KrizKs/P7_PWA_ACAA.git